Sparse-View 3D Reconstruction Pipeline using DUSt3R

Computer Vision Project by Daniel Munera Martinelli, Maria Beatriz Campos Gaiger Silveira, Leila Zanoni

Project Goal

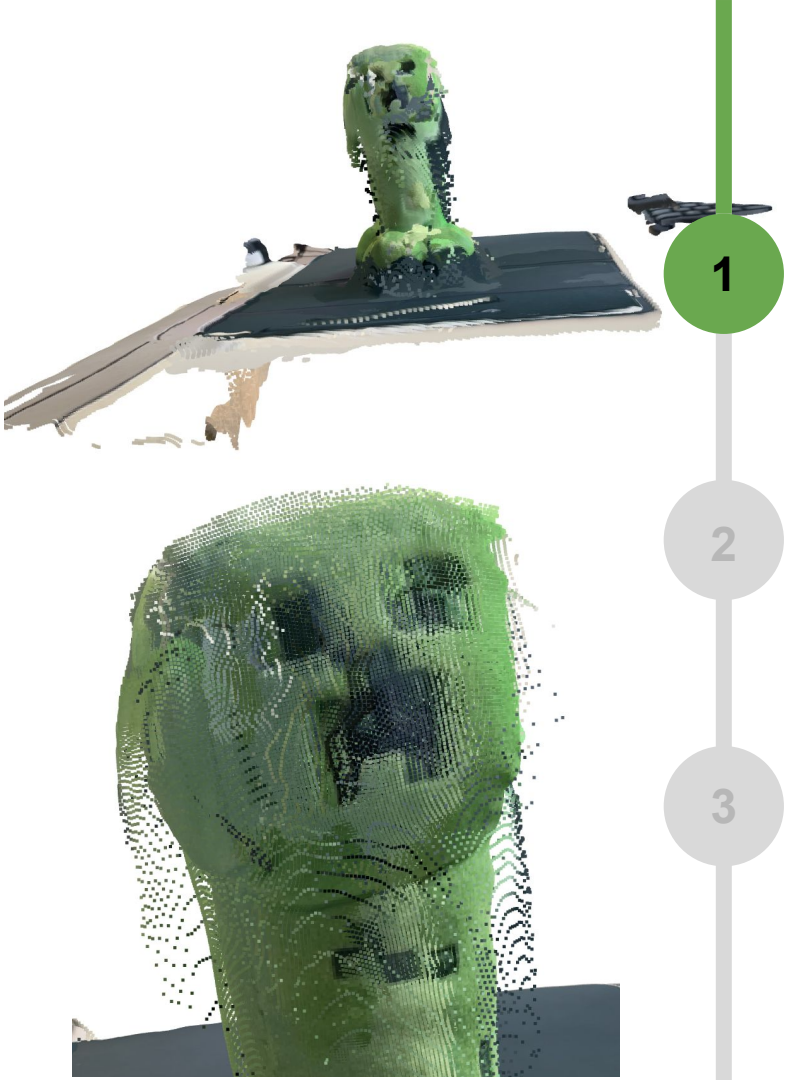
- **The Problem:** Traditional photogrammetry requires hundreds of photos and precise camera data
- **The Solution**: A lightweight, end-to-end pipeline that handles sparse inputs (10-15 images) and allows for creative artistic control
- **How**?
  - 1) Leverage AI for unconstrained 3D point cloud generation
  - 2) Cleanup of the generated point cloud
  - 3) Reconstruction of edges and object surface

Literature Overview

- **Dust3r:** It’s the core of our pipeline. Unlike COLMAP (SfM), DUSt3R doesn't need camera poses or intrinsics. It predicts "pointmaps" directly from image pairs using a Transformer architecture.
- **Other Approaches**: We also tried different technologies before sticking with Dust3r, including 3DGS (3D Gaussian Splatting) and NeRF (Neural Radiance Field), but they required much more computational resources and difficult setup (e.g. COLMAP to estimate camera positions).

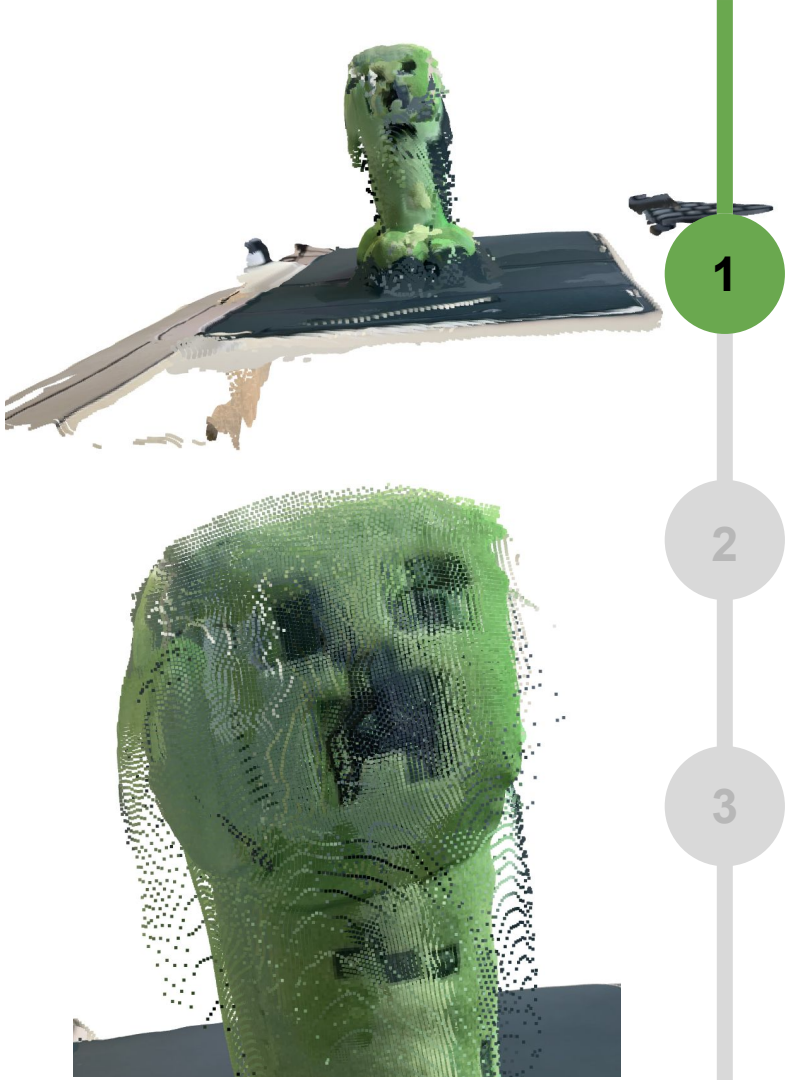
DUSt3R: Overview

The paper presents this end-to-end 3D reconstruction pipeline from un-calibrated and un-posed images, using a neural network that can regress a dense and accurate scene representation solely from a pair of images, **without prior information about the scene nor the images,** making many geometric 3D vision tasks easy.



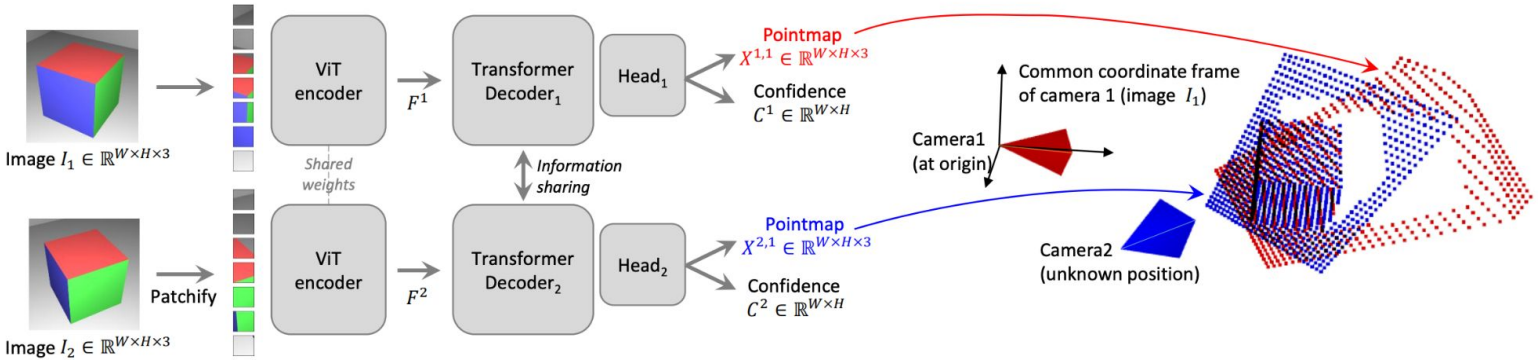
DUSt3R: Pointmaps

The key concept presented on this paper is the pointmap, which is essentially a 2D grid of 3D points. It has the same width and height as an image, but instead of storing RGB color values at each pixel, it stores **(x,y,z) coordinates** in 3D space. Using this pointmap, the network searches in 3D space to find which pixel in the first input image corresponds to which pixel in the second input image.



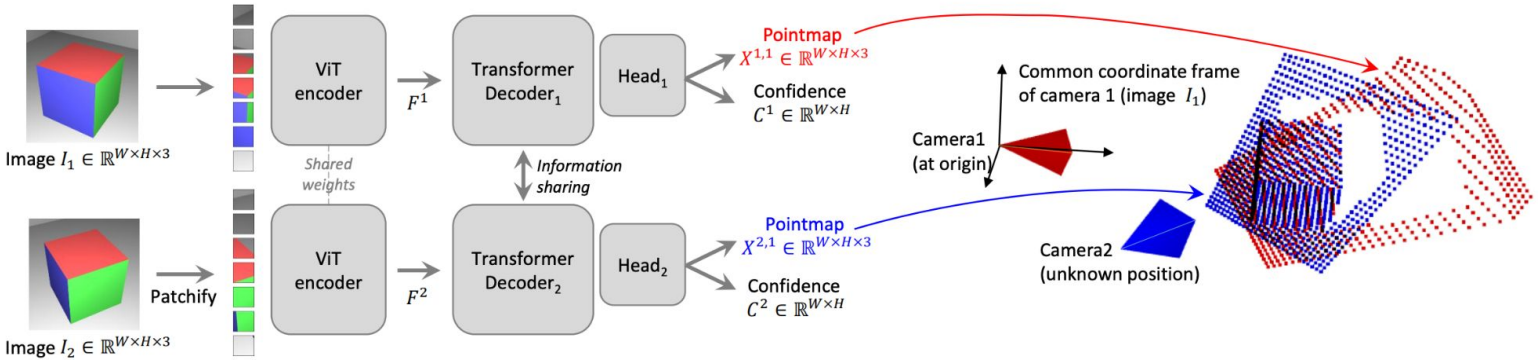
DUSt3R: Architecture

Two views of a scene are first encoded with a shared ViT encoder, then the resulting token representations are passed to two transformer decoders, that constantly exchange information via **cross-attention**. Finally, two regression heads output the corresponding **pointmaps** and associated **confidence maps**.



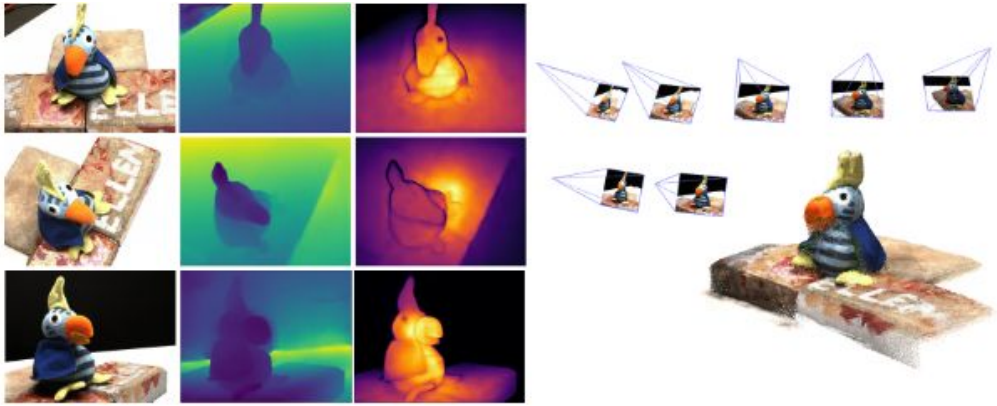
DUSt3R: Architecture

The two poitmaps are expressed in the same coordinates as the first input scene. The network is trained using a simple 3D regression loss, that is additionally **confidence-aware**, meaning that if some parts in the image are harder to predict than others, the network learns to predict a confidence score per pixel.



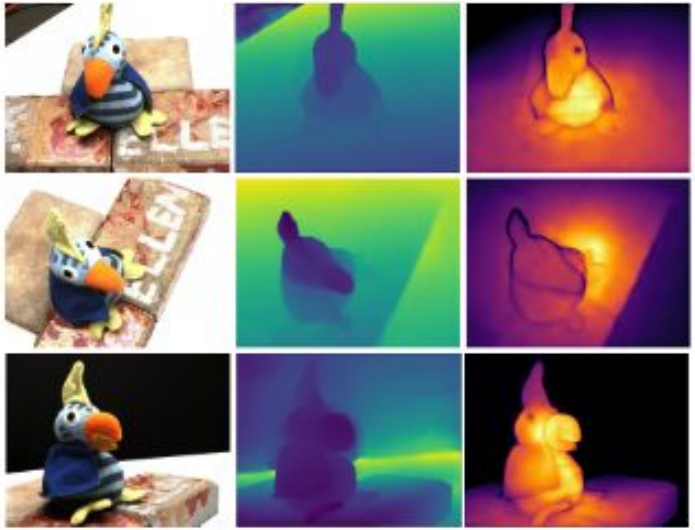
DUSt3R: Global alignment

To scale from two images to an entire scene, DUSt3R employs a global alignment step that unify pairwise predictions into a single joint 3D space. It first constructs a **connectivity graph** to identify image pairs that share visual content. Then it optimizes camera poses and geometry alignments directly in 3D space.



DUSt3R: Global alignment

By minimizing 3D projection errors using **standard gradient descent**, this optimization converges rapidly on a standard GPU, making the process easy and straightforward.



Point Cloud Cleaning

- **Statistical Outlier Removal** (SOR): Eliminating "floaters”, i.e. points that don't belong to the object

1

2

3

Point Cloud Cleaning

- **Statistical Outlier Removal** (SOR): Eliminating "floaters", i.e. points that don't belong to the object

- **RANSAC** Plane Segmentation: Identifying and removing the "table" or floor to isolate the target object

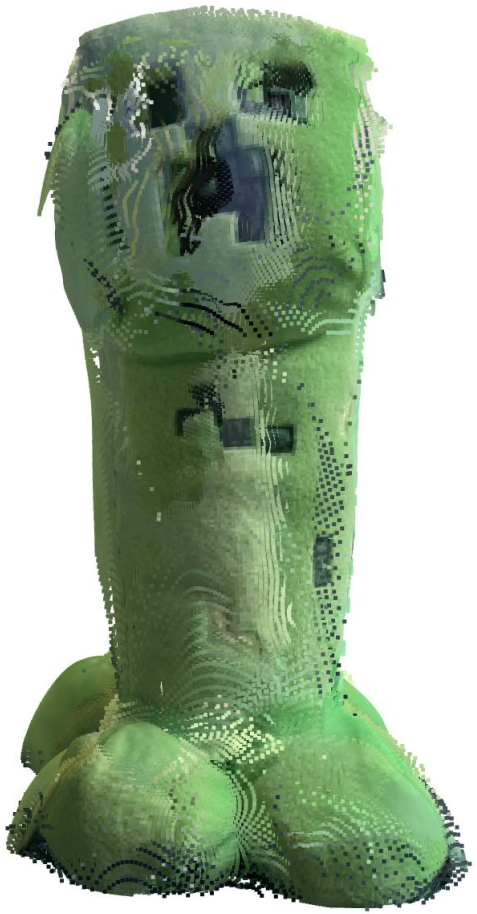
1

2

3

Point Cloud Cleaning

- **Statistical Outlier Removal** (SOR): Eliminating "floaters”, i.e. points that don't belong to the object
- **RANSAC** Plane Segmentation: Identifying and removing the "table" or floor to isolate the target object
- **DBSCAN** Object Isolation: Clustering is used to eliminate any noise left



1

2

3

Mesh Reconstruction

Before actually constructing the mesh, the point cloud needs some more processing:

- Point cloud spatial **smoothing**
- Adaptive **normal** estimation
- Poisson Surface Reconstruction
- Density-based trimming
- Connected-component cleanup

1

2

3

Mesh Reconstruction

Before actually constructing the mesh, the point cloud needs some more processing:

- Point cloud spatial smoothing
- Adaptive normal estimation
- **Poisson Surface Reconstruction**
- Density-based trimming
- Connected-component cleanup

1

2

3

Mesh Reconstruction

Before actually constructing the mesh, the point cloud needs some more processing:

- Point cloud spatial smoothing
- Adaptive normal estimation
- Poisson Surface Reconstruction
- Density-based **trimming**
- Connected-component cleanup

1

2

3

What about the bottom?

Bottom-view data is typically missing, so reconstructed meshes are **hollow at the base**. We tested multi-pass scan fusion, but it degraded quality (lower resolution, sparse polygons, more holes)

1

2

3

What about the bottom?

We therefore adopted automatic pedestal attachment: a reliable, geometry-aware way to close the base and produce presentation-ready meshes.

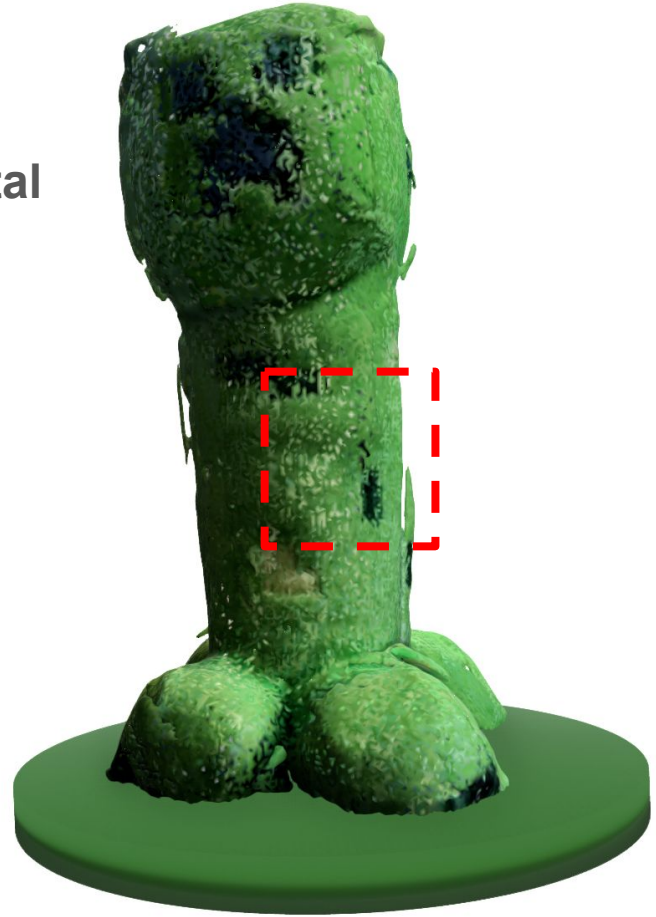
1

2

3

What about the bottom?

We therefore adopted automatic **pedestal** attachment: a reliable, geometry-aware way to close the base and produce presentation-ready meshes.



1

2

3

Effects

1 Low-poly

2 FF7 Retro

3 Voxel

4 Soft-Voxel

5 Hologram

6 PBR Material

1 Low-Poly

1. Clean Up
2. Taubin Smooth
3. Subdivision
4. Voxel Pre-Pass
5. Quadric Decimation
6. Vertex Decoupling

Effects

1 Low-poly

2 FF7 Retro

3 Voxel

4 Soft-Voxel

5 Hologram

6 PBR Material

2 FF7

Colors are quantized instead

1. Color Preservation
2. Color Stripping
3. Voxel Pre-Pass
4. Quadric Decimation
5. Color Re-Transfer
6. Vertex Decoupling
7. Color Posterization

Low-Poly

- **Less** Aggressive
- **Slower**
- **Color handling:** continuous gradients
- Smoothing and subdivision
- **Shape Fidelity:** High

FF7

- **More** Aggressive
- **Faster**
- **Color handling:** posterized
- **NO** smoothing and **NO** subdivision
- **Shape Fidelity:** Low

Effects

1 Low-poly

2 FF7 Retro

3 Voxel

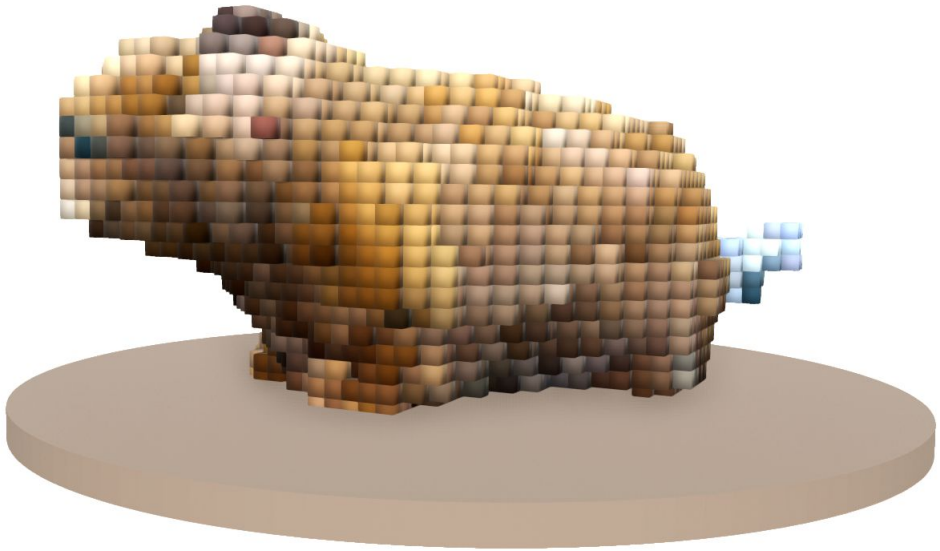
4 Soft-Voxel

5 Hologram

6 PBR Material

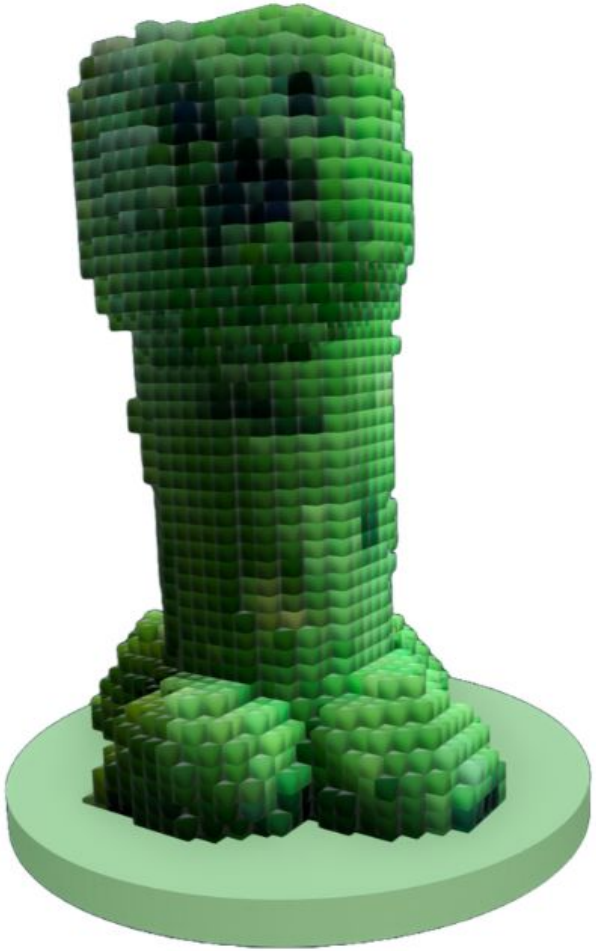
3 Voxel

1. Uniform Point Sampling
2. Voxel Grid Creation
3. Mesh Reconstruction From Voxels
4. Normals Computation



3 Voxel

1. Uniform Point Sampling
2. Voxel Grid Creation
3. Mesh Reconstruction From Voxels
4. Normals Computation



Effects

1 Low-poly

2 FF7 Retro

3 Voxel

4 Soft-Voxel

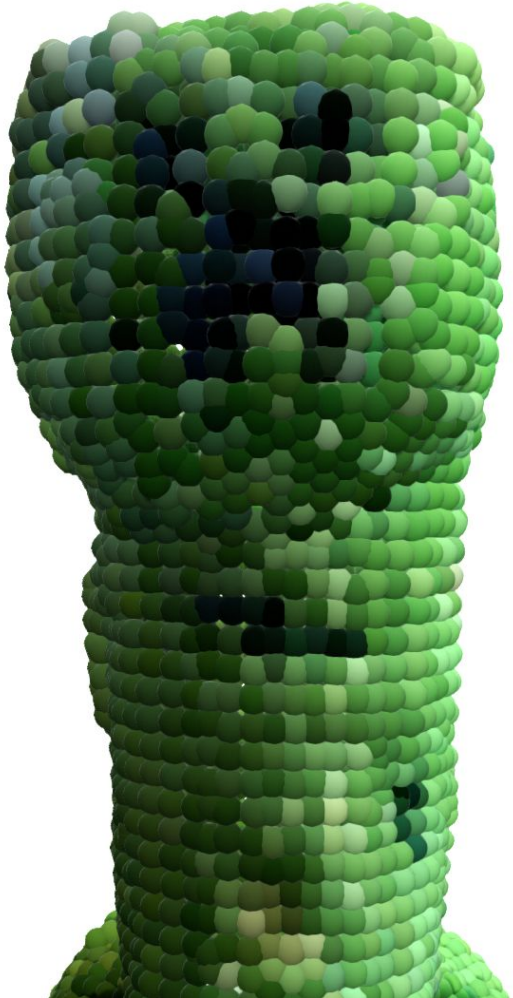
5 Hologram

6 PBR Material

4 Soft Voxel

Creates a rounded voxel effect

1. Uniform Point Sampling
2. Voxel Grid Downsampling
3. Sphere Radius Calculation
4. Sphere Creation
5. Normals Computation

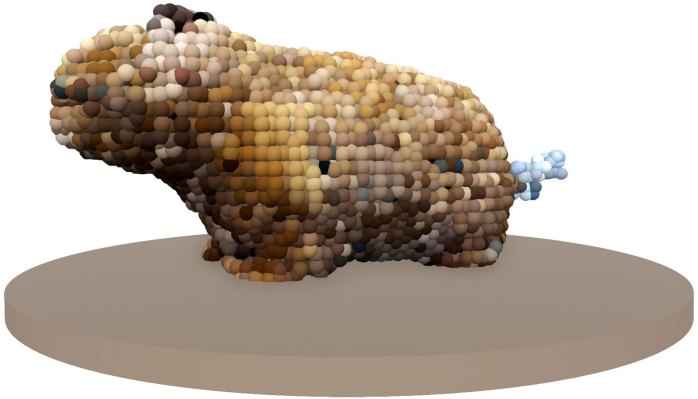


Voxel

- **Cubes**
- **Grid:** Full 3D
- **Overlap:** None
- **Polygon Count:** Moderate

Soft Voxel

- **Spheres**
- **Grid:** Sparse Downsampled Points
- **Overlap:** Yes
- **Polygon Count:** Higher



Effects

1 Low-poly

2 FF7 Retro

3 Voxel

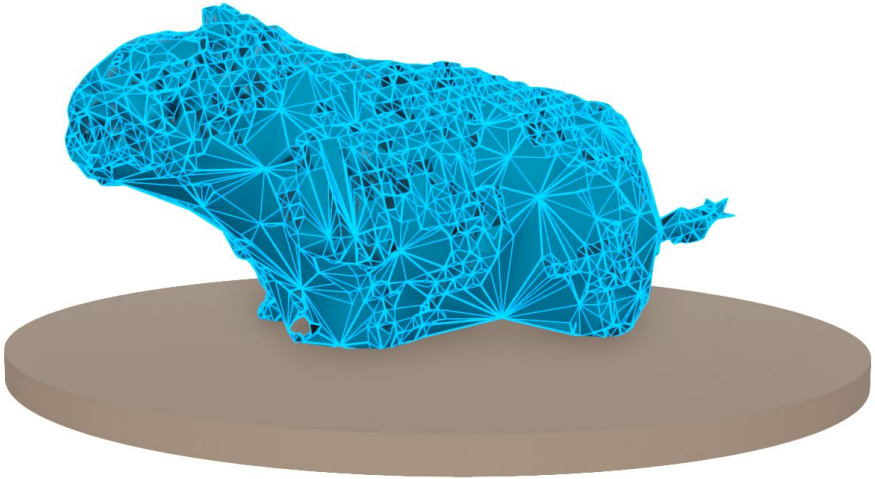
4 Soft-Voxel

5 Hologram

6 PBR Material

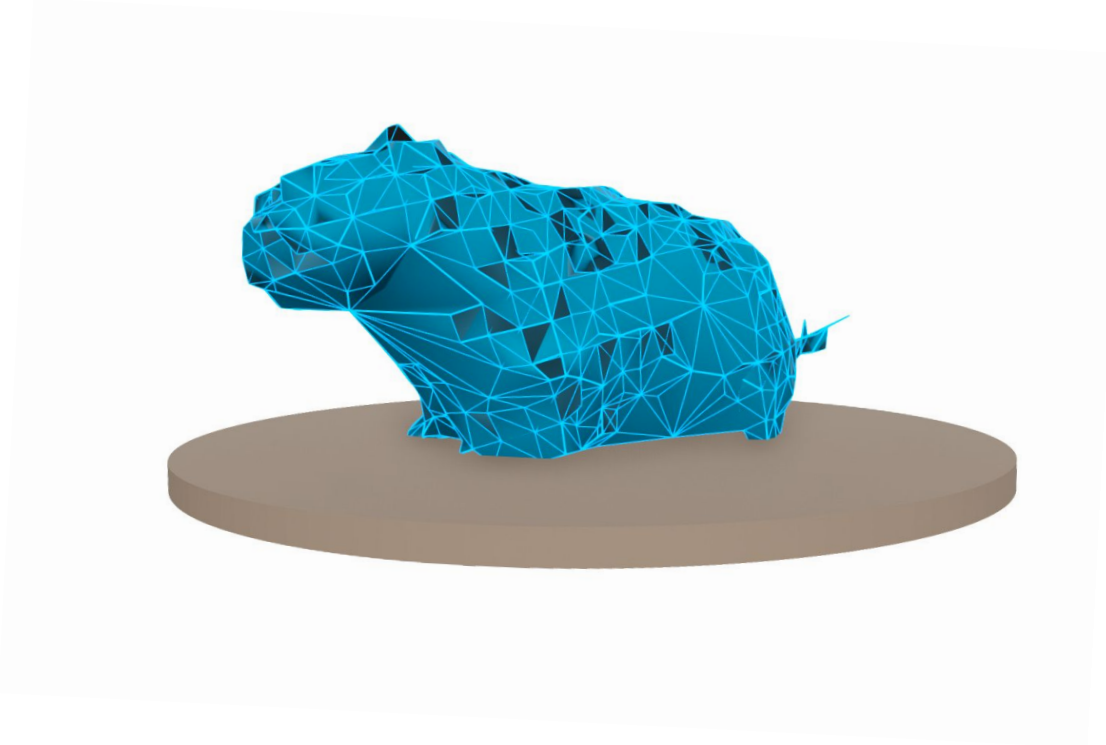
5 Hologram

1. Checks whether to simplify geometry
2. Clean and Repair Mesh
3. Create Hologram Body
4. Extract Wireframe Edges
5. Build Neon Line Overlay



5 Hologram

1. Checks whether to simplify geometry
2. Clean and Repair Mesh
3. Create Hologram Body
4. Extract Wireframe Edges
5. Build Neon Line Overlay



Effects

1 Low-poly

2 FF7 Retro

3 Voxel

4 Soft-Voxel

5 Hologram

6 PBR Material

6 PBR Material

1. Checks whether smoothing and subdivision are necessary
2. Optionally increase geometry density
3. Compute vertex normals and normalize positions
4. Calculate Base Colors
5. Real-Time Lighting Handoff
6. Write colors back to mesh

6 PBR Material

1. Checks whether smoothing and subdivision are necessary
2. Optionally increase geometry density
3. Compute vertex normals and normalize positions
4. Calculate Base Colors
5. Real-Time Lighting Handoff
6. Write colors back to mesh

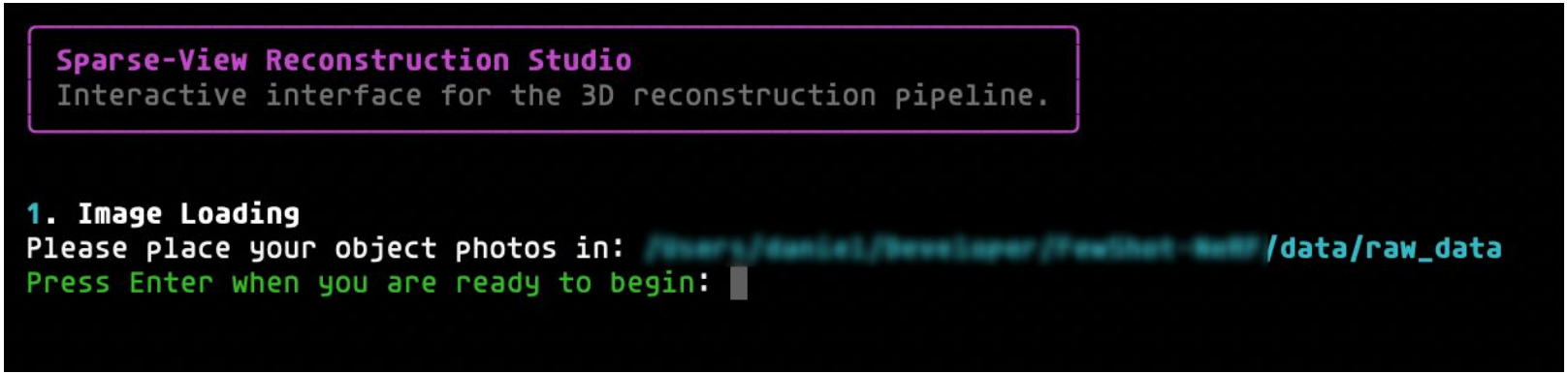
Conclusions

This project delivered an automated DUSt3R-based pipeline that **runs on mid-range hardware** in only a few minutes while supporting **real-time** mesh reconstruction and manipulation.

Although the system performs well overall, the final mesh quality can still be improved, as reconstructions may exhibit surface noise, **holes**, and occasional double-skin **artifacts**.

Conclusions

To improve usability, we also developed an interactive **CLI** that makes the full workflow easy to test and operate: after cloning the repository and completing setup, users can launch the studio interface from `src/studio.py`.



Thank you for the attention